OOP Lab Examination – 40 Marks Question Code: HDFC

Duration: 2 Hours

Problem Statement: Travel Booking Management System

Develop a Java application for a **Travel Booking Management System** for a travel company that provides **Bus Booking** and **Train Booking** services.

The system should allow the company to store booking details, calculate the total fare, apply

1. Create an Abstract Class `Booking` – 8 Marks

Create an abstract class named `Booking` with the following **instance variables**:

- `bookingId`
- `passengerName`
- `source`
- `destination`
- `numberOfPassengers`

Create a **parameterized constructor** to initialize all the above variables.

Create a concrete method:

```
displayBookingDetails()
```

to display the common booking information.

Declare the following abstract method:

```
calculateFare()
```

The method should be implemented differently by the child classes.

Also declare a **static variable**:

```
totalBookings
```

This variable should keep track of the total number of booking objects created.

---

2. Create `BusBooking` Class – 7 Marks

Create a class named `BusBooking` that extends the `Booking` class.

The class should contain the following **instance variables**:

- `busNumber`
- `busType`
- `seatNumber`
- `farePerPassenger`

Initialize all variables using a constructor.

Override the `calculateFare()` method.

The bus fare should be calculated as:

```
fare = farePerPassenger × numberOfPassengers
```

Display all bus-specific details along with the common booking details.

---

3. Create `TrainBooking` Class – 7 Marks

Create a class named `TrainBooking` that extends the `Booking` class.

The class should contain the following **instance variables**:

- `trainNumber`
- `trainName`
- `coachType`
- `berthNumber`
- `farePerPassenger`

Initialize all variables using a constructor.

Override the `calculateFare()` method.

The train fare should be calculated as:

```
fare = farePerPassenger × numberOfPassengers
```

Display all train-specific details along with the common booking details.

---

4. Create an Interface `DiscountApplicable` – 6 Marks

Create an interface named `DiscountApplicable` containing the following method:

```
calculateDiscount(double fare)
```

Both `BusBooking` and `TrainBooking` classes should implement this interface.

Apply different discount rules:

Bus Booking:

- If the number of passengers is 4 or more, provide a **10% discount**.
- Otherwise, no discount.

Train Booking:

- If the number of passengers is 3 or more, provide a **5% discount**.
- Otherwise, no discount.

The method should return the discount amount.

---

5. Static Variable – 3 Marks

The static variable `totalBookings` in the `Booking` class should be incremented whenever a new booking object is created.

At the end of the program, display:

```
Total Bookings Created: <value>
```

The value should represent the total number of `BusBooking` and `TrainBooking` objects created.

---

6. Main Method and Object Creation – 9 Marks

Create a class `TravelBookingDemo` containing the `main()` method.

Create at least:

- **2 BusBooking objects**
- **2 TrainBooking objects**

Store the objects using appropriate reference variables.

For every booking, display:

1. Booking ID
2. Passenger name
3. Source
4. Destination
5. Number of passengers
6. Bus/Train-specific details
7. Fare
8. Discount
9. Final payable amount

The final payable amount should be calculated as:

```
Final Amount = Total Fare - Discount
```

Finally, display the value of the static variable:

```
totalBookings
```

Java Lab Examination – ArrayList **Question Code: AXIS**

Problem Statement: Vehicle Service Management System
Maximum Marks: 40
Duration: 2 Hours

Develop a Java application for a **Vehicle Service Management System** using `ArrayList`.

Create a class `VehicleService` with the following **instance variables**:

- `serviceId`
- `vehicleNumber`
- `ownerName`
- `vehicleType`
- `serviceType`
- `serviceCost`
- `serviceStatus`

Use an `ArrayList<VehicleService>` to store all service records.

1. Add and Display Service Records – 8 Marks

Implement the following methods:

```
addService()
displayAllServices()
```

`addService()` should create a `VehicleService` object and add it to the `ArrayList`.

The `serviceId` must be unique.

`displayAllServices()` should display all service records in a meaningful format.

Possible values:

Vehicle Type:

- Car
- Bike
- SUV

Service Type:

- General Service
- Oil Change
- Repair
- Full Service

Service Status:

- Pending
- In Progress
- Completed

---

2. Search Service Record – 5 Marks

Implement:

`searchByVehicleNumber()`

The user should enter a vehicle number.

Display the complete service details of the matching vehicle.

If no record is found, display an appropriate message.

The search must be performed using the `ArrayList`.

---

3. Update Service Record – 5 Marks

Implement:

`updateService()`

The user should enter a `serviceId`.

If the service record exists, allow the user to update:

- `serviceType`
- `serviceCost`
- `serviceStatus`

If the service ID does not exist, display an appropriate message.

---

4. Remove Service Record – 4 Marks

Implement:

`removeService()`

The user should enter a `serviceId`.

If the record exists, remove it from the `ArrayList`.

Otherwise, display:

```
Service record not found.
```

---

5. Sort Service Records – 5 Marks

Implement:

```
sortByServiceCost()
```

Sort the service records based on `serviceCost` in **descending order** using `Comparator`.

After sorting, display the records.

The most expensive service should appear first.

---

6. Business Logic – 8 Marks

Implement:

```
generateServiceReport()
```

The method should analyze all service records and display:

1. **Total revenue generated from completed services**
2. **Vehicle number of the service having the highest service cost**

Java Lab Examination – File Handling **Question Code: ICICI**

Problem Statement: Courier Package Management System
Maximum Marks: 40
Duration: 2 Hours

Develop a Java application for a **Courier Package Management System** using **object-based file handling**.

Create a class `CourierPackage` that implements `Serializable`.

The class should contain the following instance variables:

- `packageId`
- `senderName`
- `receiverName`
- `source`
- `destination`
- `packageWeight`
- `deliveryCharge`
- `deliveryStatus`

Possible values for `deliveryStatus`:

- Booked
- In Transit
- Delivered
- Cancelled

---

1. Create CourierPackage Class – 5 Marks

Create a class named:

`CourierPackage`

The class must:

- Implement `Serializable`
- Contain all the required instance variables
- Have a parameterized constructor
- Provide a method to display package details

Each package must have a unique `packageId`.

---

2. Add and Write Object to File – 7 Marks

Create a method:

`addPackage()`

The method should:

1. Accept package details from the user.
2. Create a `CourierPackage` object.
3. Write object into the file.

Use:

```
ObjectInputStream
ObjectOutputStream
```

The data must remain available even after the application is closed.

---

3. Read and Display Objects – 5 Marks

Create a method:

`displayAllPackages()`

Read all `CourierPackage` objects from the file and display their details.

If there are no records, display an appropriate message.

---

4. Search Package – 5 Marks

Create a method:

`searchPackage()`

The user should enter a `packageId`.

Read the objects from the file and search for the matching package.

If found, display the complete package details.

If not found, display:

`Package not found.`

---

5. Update Package – 5 Marks

Create a method:

`updatePackage()`

The user should enter a `packageId`.

If the package exists, allow the user to update:

- `destination`
- `deliveryCharge`
- `deliveryStatus`

After updating, rewrite the complete collection to the file.

If the package ID does not exist, display an appropriate message.

---

6. Delete Package – 4 Marks

Create a method:

`deletePackage()`

The user should enter a `packageId`.

If the package exists:

- Remove the corresponding object.
- Rewrite the remaining objects to the file.

If the package does not exist, display:

`Package not found.`

---

7. Business Logic – 5 Marks

Create a method:

`generateDeliveryReport()`

Read all package objects from the file and display:

1. Total delivery revenue from Delivered packages.
2. The package having the **highest delivery charge**.

Display the report in a meaningful format.

Example:

```
===== DELIVERY REPORT =====

Delivered Packages       : 8
Total Delivery Revenue   : ₹18,500
Highest Charge Package   : P105
Highest Delivery Charge  : ₹2,500
```

The calculations must be performed using the objects read from the file.

8. Main Method and Menu – 4 Marks

Create a class:

```
CourierManagement
```

with the `main()` method.

Display the following menu:

```
===== COURIER PACKAGE MANAGEMENT =====

1. Add Package
2. Display All Packages
3. Search Package
4. Update Package
5. Delete Package
6. Generate Delivery Report
7. Exit
```

The menu should continue until the user selects **Exit**.

Sample Data

Students may use the following data:

ID	Sender	Receiver	Source	Destination	Weight	Charge	Status
P101	Arun	Ravi	Bangalore	Chennai	2.5	450	Delivered
P102	Meena	Kiran	Delhi	Mumbai	5.0	900	In Transit
P103	Raj	Priya	Bangalore	Hyderabad	1.5	350	Delivered
P104	Suresh	Anu	Pune	Bangalore	8.0	1400	Delivered
P105	Kumar	Divya	Chennai	Delhi	10.0	2500	In Transit

JDBC Lab Examination **Question Code: SBI**

Problem Statement: Vehicle Rental Management System
Maximum Marks: 40
Duration: 2 Hours

Develop a **Java JDBC application** for a **Vehicle Rental Management System**.

A vehicle rental company maintains details of vehicles available for rental. The application should allow the user to add, modify, delete, search, and display vehicle records using **JDBC and MySQL**.

Create a database named:

`vehicle_rental`

Create a table named:

`vehicle`

with the following columns:

Column	Data Type	Description
`vehicle_id`	INT	Primary Key
`vehicle_number`	VARCHAR(20)	Unique vehicle number
`vehicle_model`	VARCHAR(50)	Vehicle model
`vehicle_type`	VARCHAR(30)	Car / Bike / SUV
`rental_rate`	DOUBLE	Rental charge per day
`available_days`	INT	Number of days available
`status`	VARCHAR(20)	Available / Rented

---

1. Database and Table Creation – 4 Marks

Create the `vehicle_rental` database and `vehicle` table.

Set `vehicle_id` as the **Primary Key**.

2. Insert Vehicle – 6 Marks

Implement a method:

`addVehicle()`

Accept vehicle details from the user and insert a new record into the `vehicle` table.

The application should use `PreparedStatement` for the INSERT operation.

Before insertion, ensure that the `vehicle_number` is not already registered.

Display an appropriate success or error message.

---

3. Update Vehicle – 5 Marks

Implement:

`updateVehicle()`

Ask the user to enter `vehicle_id`.

If the vehicle exists, allow the user to update:

- `vehicle_model`
- `rental_rate`
- `available_days`
- `status`

Use a parameterized `UPDATE` query.

Display an appropriate message if the specified vehicle ID does not exist.

---

4. Delete Vehicle – 5 Marks

Implement:

`deleteVehicle()`

Ask the user to enter `vehicle_id`.

If the vehicle exists, delete the corresponding record from the database.

If the ID does not exist, display:

```
Vehicle not found.
```

Use a parameterized `DELETE` query.

---

5. Select Vehicle by ID – 5 Marks

Implement:

```
findVehicleById()
```

Ask the user to enter `vehicle_id`.

Retrieve and display the complete details of the vehicle.

If no record exists for the given ID, display an appropriate message.

Use:

```
PreparedStatement
ResultSet
```

---

6. Select All Vehicles – 5 Marks

Implement:

```
displayAllVehicles()
```

Retrieve all vehicle records from the database and display them in a meaningful tabular format.

7. Main Method and Menu – 4 Marks

Create a class:

```
VehicleRentalManagement
```

containing the `main()` method.

Display the following menu:

```
===== VEHICLE RENTAL MANAGEMENT =====

1. Add Vehicle
2. Update Vehicle
3. Delete Vehicle
4. Find Vehicle by ID
5. Display All Vehicles
6. Calculate Rental Estimate
```

```
7. Exit
```

The program should continue displaying the menu until the user selects **Exit**.

Sample Data

Students may insert the following records for testing:

ID	Vehicle Number	Model	Type	Rate	Days	Status
101	KA01AB1234	Swift	Car	1800	10	Available
102	KA05CD5678	Activa	Bike	800	15	Rented
103	KA03EF4321	Creta	SUV	2500	7	Available
104	KA04GH9876	Baleno	Car	1600	5	Available
105	KA02JK4567	Royal Enfield	Bike	1200	12	Rented